

LAPORAN PRAKTIKUM

Praktikum 5 - ABSTRACT CLASS & INTERFACE

Mata Kuliah Pemrograman Berorientasi Objek · D3 Teknik Informatika

Identitas	Isian
Nama	Rizky Yanuar Irawan
NIM	251511029
Kelas	2A D3
Tanggal Mulai	07 Oktober 2026
Tanggal Pengumpulan	07 Oktober 2026
URL Repository	https://github.com/RizkyYanuar/pbo-praktikum5.git

1. Praktikum I - Employee, Manager, dan Sortable

1. Step 1 - Bangun Circle employee

- Checkpoint 1: Mana state Employee? Mana behavior Employee? Mengapa salary tetap dibuat private walaupun nanti Employee akan mempunyai subclass?

State employee adalah name, salary, hireDay, hireMonth, hireYear

Behavior employee adalah print(), raiseSalary(), hireYear(), getSalary()

Salary tetap dibuat private agar employee tetap menjadi pihak utama yang berhak memanipulasi/mengontrol bagaimana salary diubah (enkapsulasi).

2. Step 2 - Uji beberapa object Employee

- Perbandingan output sebelum dan sesudah kenaikan salary

```
61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws\
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993
Sesudah kenaikan 5% gaji:
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

Ketiga objek memiliki nilai gaji yang berbeda

3. Step 3 - Buat Manager sebagai subclass Employee

- Checkpoint 2: Mengapa constructor Manager memanggil super(...)? Pada raiseSalary(), mengapa Manager tetap memanfaatkan super.raiseSalary(...) daripada menulis ulang seluruh proses perubahan salary?

Manager tetap memanfaatkan super.raiseSalary() karena yang memiliki akses pada state salary adalah superclassnya, sehingga yang bertugas melakukan perubahan adalah behavior superclassnya, bukan langsung dari subclass itu sendiri.

4. Step 4 - Uji Employee dan Manager dalam satu program

- Hasil Eksekusi

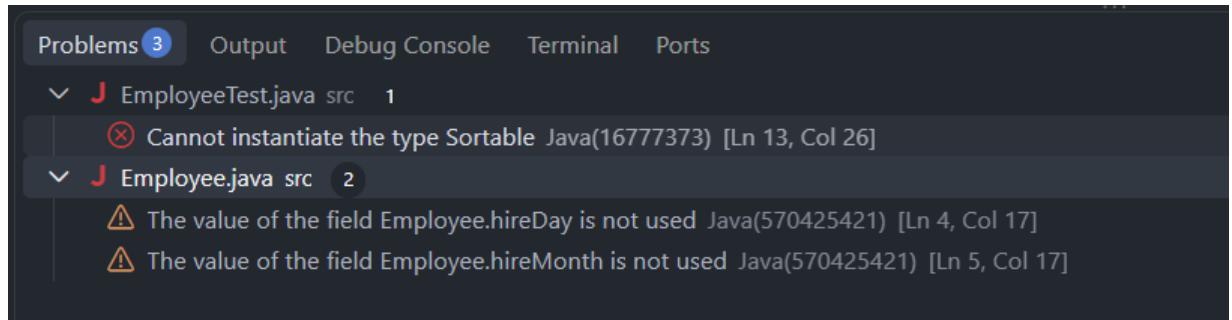
```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan5'
-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanu
61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws\pertemuan5_85f990b4\bin' 'EmployeeTe
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

Dapat dilihat bahwa subclass manager memiliki mekanisme kenaikan gaji yang berbeda dengan employee biasa. Ini dikarenakan subclass manager melakukan overriding behavior superclassnya dan melakukan perhitungan kenaikan gaji yang berbeda dimana kenaikan gaji di manager dihitung berdasarkan (persen kenaikan gaji + (0.5 * lama dipekerjakan)).

- b. Checkpoint 3: Lengkapi kalimat dengan bahasa Anda: Manager adalah sebuah subclass dari employee sehingga object Manager dapat ditempatkan pada array Employee.

5. Step 5 - Buat abstract class Sortable

- a. Hasil pesan error



- b. Checkpoint 4: Sortable berguna sebagai class untuk menetapkan kewajiban compare(...), tetapi tidak memiliki bentuk object konkret yang perlu dibuat langsung. Apa bukti compiler yang Anda dapatkan?

Bukti compiler yang saya dapatkan adalah Ketika mencoba membuat objek Sortable secara langsung di file test, muncul error cannot instantiate the type Sortable. Hal ini menunjukkan bahwa Sortable tidak bisa digunakan secara langsung Ketika class belum memiliki implementasi untuk compare() nya dan belum memakai keyword extends.

6. Step 6 - Jadikan Employee subclass dari Sortable

- a. Checkpoint 5: Apa yang diwajibkan Sortable kepada Employee? Mana bagian yang hanya deklarasi pada superclass abstract, dan mana bagian yang menjadi implementasi konkret di Employee?

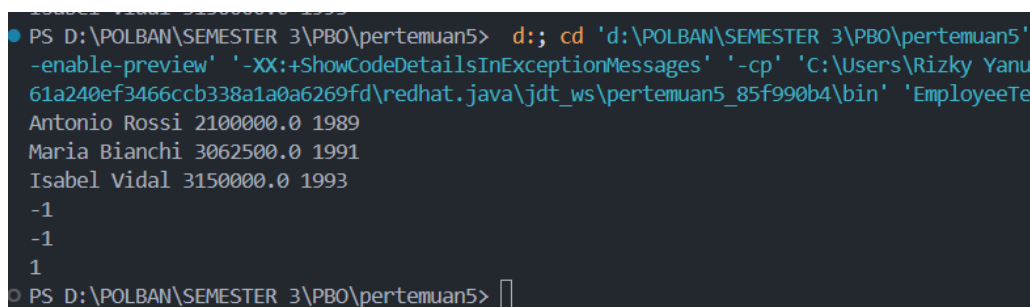
Yang diwajibkan sortable adalah implementasi dari compare() di class Employee. Bagian yang hanya deklarasi pada superclass abstract ada pada file Sortable.java itu sendiri sedangkan implementasi konkret di employeenya adalah bagian @override behavior compare().

7. Step 7 - Uji compare(...)

- a. Prediksi Hasil

Hasil perbandingan staff[0] dan staff[1] akan mengembalikan -1, hasil perbandingan staff [1] dan staff[2] akan mengembalikan -1, hasil perbandingan staff[2] dan staff [0] akan mengembalikan 1

- b. Hasil Aktual



Staff[0] dengan [1] menghasilkan -1 karena besaran gaji daripada staff [0] lebih kecil dari staff [1], hal yang sama berlaku untuk staff[1] dan staff[2] dimana besaran gaji staff[1] lebih kecil daripada

staff[2], sedangkan untuk perbandingan antara staff[2] dan staff[0] menghasilkan 1 karena besaran gaji staff[2] lebih besar daripada besaran gaji staff[0]

8. Step 8 - Uji inheritance chain dan batas extends

a. Eksperimen A

```
4 public class Manager extends Employee extends Sortable {  
5     private String secretaryName;  
6  
7     public Manager(String name, double salary, int day, int month, int year) {  
8         super(name, salary, day, month, year);  
9         secretaryName = "";  
10    }
```

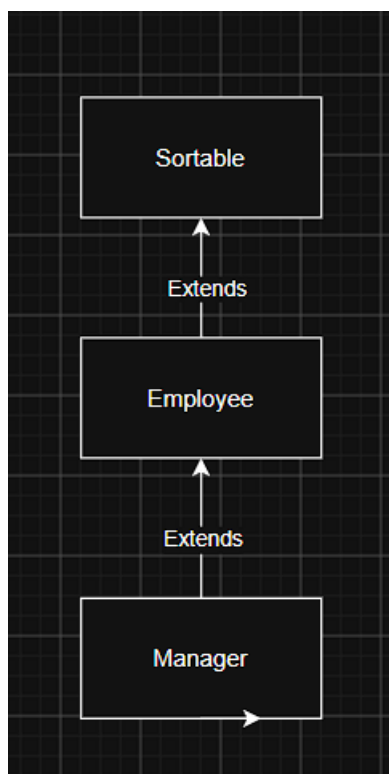
Syntax error on token "extends", permits expected Java(1610612940)
View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

b. Eksperimen B

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> d:; cd '  
-enable-preview' '-XX:+ShowCodeDetailsInException  
61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws\  
-1  
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> |
```

c. Checkpoint 6: Mengapa Manager tetap memiliki compare(...) walaupun tidak pernah menulis 'extends Sortable'? Gambarkan rantai class: Sortable <- Employee <- Manager.

Manager tetap memiliki compare() karena manager merupakan subclass dari employee dan employee merupakan subclass dari sortable.



2. Praktikum II - Goods dan Taxable

9. Step 9 - Pahami model sebelum coding

- State apa yang cukup umum sehingga berada di Goods?
State yang cukup umum adalah description dan price.
- State apa yang hanya dimiliki Food, Toy, dan Book?
Food memiliki state calories, toy memiliki state minimumAge, book memiliki state author
- Mengapa Taxable tidak diterapkan pada semua subclass Goods pada model ini?
Karena memiliki aturan sendiri yaitu makanan tidak dikenakan pajak

10. Step 10 - Buat class Goods

- Checkpoint 8: Mengapa description dan price diletakkan di Goods, bukan ditulis ulang di Food, Toy, dan Book?
Karena setiap instance dari goods sudah dipastikan memiliki description dan price, sehingga kedua state tersebut diletakkan di class goods agar lebih efisien

11. Step 11 - Buat Food sebagai subclass Goods

- Hasil

```
public class GoodsTest {  
    Run | Debug  
    public static void main(String[] args) {  
        Goods g1 = new Goods(description: "Laptop", price: 1500.0);  
        g1.display();  
    }  
}
```

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> &  
ionMessages' '-cp' 'C:\Users\Rizky Yanuar I  
ws\pertemuan5_85f990b4\bin' 'GoodsTest'  
Description : Laptop  
Price : 1500.0  
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

12. Step 12 - Buat interface Taxable

- Hasil error dari percobaan

```
public class GoodsTest {  
    Run | Debug  
    public static void m  
        Goods g1 = new G  
        g1.display();  
    Taxable t = new Taxable() {  
        }  
}
```

Cannot instantiate the type Taxable Java(16777373)
Taxable
Source: pertemuan5_85f990b4
View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

- Checkpoint 8: Apa persamaan hasil eksperimen 'new Sortable()' dan 'new Taxable()'? Apa perbedaan bentuk deklarasi Sortable dan Taxable?
Keduanya tidak bisa di instansiasi menjadi objek secara langsung, perbedaannya terletak pada deklarasinya. Sortable merupakan abstract class sedangkan taxable merupakan interface, sortable di implementasikan menggunakan extends sedangkan taxable menggunakan implements.

13. Step 13 - Buat Toy yang extends Goods dan implements Taxable

- a. Checkpoint 9: Pada deklarasi 'Toy extends Goods implements Taxable', bagian mana yang menyatakan "Toy adalah Goods" dan bagian mana yang menyatakan "Toy memenuhi kontrak Taxable"?

Bagian yang menyatakan toy adalah goods adalah "toy extends goods" yang berarti mainan adalah barang, sedangkan untuk bagian yang menyatakan toy memenuhi kontrak taxable adalah "implements taxable" yang berarti toy adalah goods yang memiliki kemampuan untuk dikenakan pajak

14. Step 14 - Buat Book yang extends Goods dan implements Taxable

- a. Hasil Perbandingan Book dan Toy

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan5\src\com\redhat\java\jdt_ws\pertemuan5'
-compile -enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages 61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws\pertemuan5
Description : Laptop
Price : 1500.0
Description : Action Figure
Price : 20.0
Minimum age : 5
Tax : 1.2
Description : Java Programming
Price : 50.0
Author : John Doe
Tax : 3.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

Hasilnya adalah toy memiliki state khusus yaitu minimum age, sedangkan book memiliki author. State sisanya sama karena toy dan book memiliki superclass yang sama, yaitu goods dan kemampuan yang sama yaitu taxable.

15. Step 15 - Uji hierarchy dan contract secara utuh

- a. Output

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan5\src\com\redhat\java\jdt_ws\pertemuan5'
-compile -enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages 61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws\pertemuan5
Description : Bread
Price : 15000.0
Calories : 250

Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0

Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0

Toy tax : 7200.0
Book tax : 5400.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

Dapat dilihat bahwa food tidak memiliki tax, dikarenakan food tidak mengimplementasikan interface taxable.

16. Step 16 - Sengaja langgar kontrak interface

a. Hasil

```
Description : Robot
Price : 120000.0
Minimum age : 8
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The type Toy must implement the inherited abstract method Taxable.calculateTax()

    at Toy.calculateTax(Toy.java:1)
    at Toy.display(Toy.java:22)
    at GoodsTest.main(GoodsTest.java:15)
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

b. Yang diminta compiler dari toy adalah class toy harus mengimplementasikan method yang diwariskan dari taxable, yaitu calculateTax().

c. Setelah diperbaiki

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5> d;; cd
-enable-preview' '-XX:+ShowCodeDetailsInExceptio
61a240ef3466ccb338a1a0a6269fd\redhat.java\jdt_ws
Description : Bread
Price : 15000.0
Calories : 250

Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0

Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0

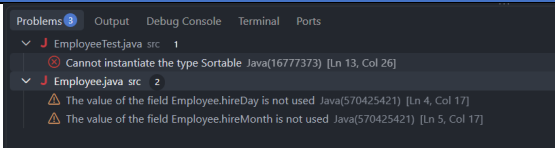
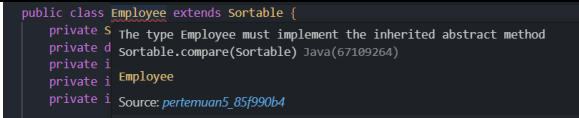
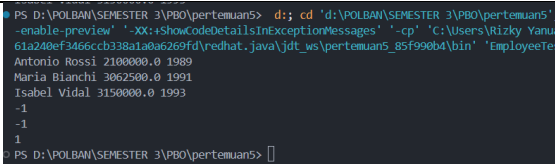
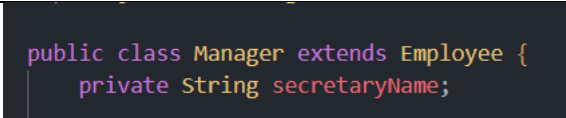
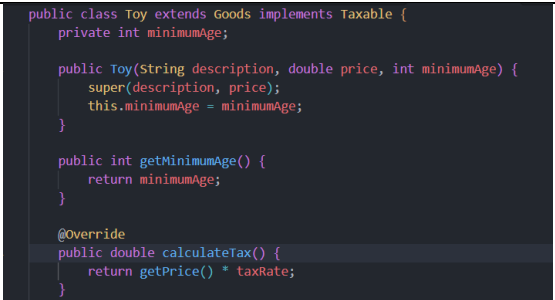
Toy tax : 7200.0
Book tax : 5400.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan5>
```

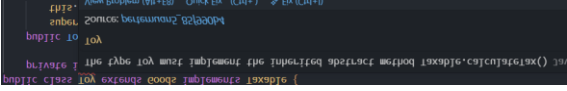
17. Step 17 - Bandingkan abstract class dan interface dari pengalaman code

Pertanyaan	Sortable (abstract class)	Taxable (Interface)
Dapat dibuat object langsung?	Tidak	Tidak
Menggunakan extends atau implements?	Menggunakan extends	Implements
Dapat mempunyai implementasi method konkret?	Dapat	Tidak

Peran utamanya pada kasus praktikum?	Untuk membandingkan besaran gaji antar objek	Untuk menghitung jumlah pajak yang dikenakan terhadap suatu barang
Apakah class dapat mempunyai parent class lain sekaligus?	Tidak	boleh

18. Step 18 - Uji sebelum menyatakan selesai

ID	SKENARIO	EXPECTED RESULT	STATUS	BUKTI
T5-01	new Sortable() dicoba	Compiler menolak instansiasi abstract class	PASS	
T5-02	Employee extends Sortable tanpa compare()	Compiler menuntut implementasi abstract method	PASS	
T5-03	Employee mengimplementasikan compare()	Hasil perbandingan -1/0/1 sesuai salary	PASS	
T5-04	Manager hanya extends Employee	Manager tetap memiliki compare() melalui inheritance chain	PASS	
T5-05	Manager extends Employee extends Sortable	Compiler menolak multiple extends class	PASS	
T5-06	Toy/Book implements Taxable	calculateTax() dapat dipanggil	PASS	

T5-07	calculateTax() dihapus dari Toy	Compiler menolak class konkret yang tidak memenuhi contract	PASS	
T5-08	Food diuji sebagai Goods	State/behavior Goods tersedia melalui inheritance	PASS	<pre>public class Food extends Goods { private int calories; public Food(String description, double price, int calories) { super(description, price); this.calories = calories; } public int getCalories() { return calories; } @Override public void display() { super.display(); System.out.println("Calories : " + calories); } }</pre>

19. Self Assesment

Pernyataan	Skor 1-5
Saya memahami mengapa abstract class tidak dapat diinstansiasi.	5
Saya dapat membuat abstract method dan implementasinya di subclass.	5
Saya memahami inheritance chain tanpa menulis extends lebih dari satu class.	5
Saya dapat membuat interface dan class yang implements interface tersebut.	5
Saya memahami bahwa implements berarti memenuhi contract method.	5
Saya dapat membedakan kapan konsep lebih cocok menjadi abstract class dan kapan menjadi interface.	5

20. Refleksi

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Sortable digunakan sebagai class dasar yang mewajibkan implementasi compare pada subclassnya. Berguna dikarenakan memiliki fungsi yang jelas yaitu membandingkan besaran gaji antar objek employee

2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Compiler akan meminta implementasi compare() dan menampilkan pesan error

3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Karena manager merupakan subclass dari employee, sedangkan employee adalah subclass dari sortable.

4. Mengapa `class Manager extends Employee extends Sortable` tidak diperlukan dan tidak valid di Java?

Karena di java, sebuah class tidak bisa menerapkan multiple inheritance, cukup extends employee saja karena employee sudah extends sortable

5. Apa yang dijanjikan Toy ketika menulis `implements Taxable`?

Toy berjanji akan mengimplementasikan calculateTax() didalamnya

6. Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

Karena food, toy, dan book adalah barang, tetapi tidak semua subclass barang pada model ini membutuhkan perhitungan pajak.

7. Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

Interface mendefinisikan bahwa setiap class yang mengimplementasikannya akan memiliki sebuah behavior tertentu, namun untuk pengimplementasiannya akan berbeda antar objek. Sehingga apabila sebuah class menulis implements taxable tetapi tidak mengimplementasikan calculateTax() maka akan menghasilkan error

8. Lengkapi dengan bahasa Anda sendiri: "Abstract class membantu saya ketika ..., sedangkan interface membantu saya ketika ..."

Abstract class membantu saya Ketika beberapa class memiliki state dan behavior yang sama, interface membantu saya ketika membutuhkan behavior atau contract yang sama tetapi tidak harus berada dalam satu inheritance yang sama